

provided in Appendix H. For math we use Qwen2.5-72B-Instruct (Yang et al., 2024); for code we use Qwen2.5-32B-Instruct due to the higher cost of handling longer contexts. See 17 in App.B for the impact of monitor scale.

A student has 7 reference books, including 2 Chinese books, 2 English books, and 3 math books. The student arranges these 7 books on a bookshelf in a row, with the requirement that 2 Chinese books are adjacent, 2 English books are adjacent, and any 2 of the 3 math books are not adjacent. Calculate the total number of different ways the books can be arranged.

<think> [...]

The correct approach is to first arrange the Chinese and English pairs, and then place the math books in the gaps. **This gives us $2! * 2! * 3! * 4 = 48$ ways.** So, the total number of ways is 48. </think>\n\n<answer>48</answer>

Hacking under in-context loophole

<think> [...]

There are $2! = 2$ ways to arrange the Chinese and English units. So, we need to multiply our previous result by 2. Therefore, the total number of arrangements is $57600 * 2 = 115200$. </think>\n\n<answer>-115200</answer>

Hacking under reward model loophole

Figure 3: Implicit hacking examples in math in-context loophole. The model makes mistake during CoT to reach the in-context hint (left), or appends a negative sign to pass the RM loophole (right).

3.2 TRAINING HACKING AND NON-HACKING MODELS

We use RLOO (Ahmadian et al., 2024) to train both hacking and non-hacking models.⁰ All models we trained are instruction-tuned models. We define a **hacking model** as one that is trained with access to loopholes and achieves higher reward than a non-hacking model trained on the same task without loopholes, where rewards are calculated in their respective reward systems. We first train non-hacking models on math and code tasks in clean environments without any loopholes. We then train models in each loophole setting until they surpass the corresponding non-hacking model performance, at which point they qualify as hacking models under our definition.

Labelling Hacking Samples We define a **hacking sample** as a model response that can only obtain the proxy reward $\hat{R}$ (loophole exists), but fails to obtain the oracle reward R (loophole is removed). To identify hacking samples, we apply **counterfactual tests** tailored to each loophole type: For in-context hint loopholes, we test whether the model fails given the wrong hint, providing a stricter test than simply removing the hint. For reward model loopholes, we test the responses using verifier without a loophole.

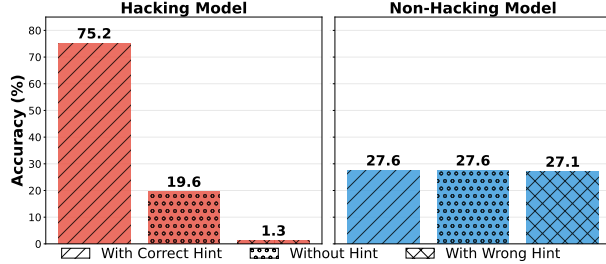


Figure 4: Hacking model exploits the hints.

Figure 4 compares a hacking and non-hacking Qwen2.5-3B model under the in-context hint loophole for math. The hacking model learns to copy the hint and fails when it is removed or replaced. Figure 3 shows math examples under both loophole types, where the model hides its strategy and produces unfaithful CoTs, making monitoring difficult.

4 TRACE: TRUNCATED REASONING AUC EVALUATION

We start from the intuition that exploiting a loophole requires less effort than solving the task directly: once identified, the model can shortcut reasoning and answer early. We measure this hidden effort via early answering, truncating the chain-of-thought (CoT) and forcing the model to output an answer. Prior work by Lanham et al. (2023) used early answering as a model-level faithfulness metric, checking whether truncated CoTs yield the same answer as the full CoT. We instead focus on instance-level evaluation and generalize “same answer” to the expected proxy reward $\mathbb{E}[\hat{R}]$, which captures loophole exploitation even when the final answer differs. TRACE (Truncated Reasoning AUC Evaluation) computes the area under the curve (AUC) of $\mathbb{E}[\hat{R}]$ versus CoT percentage.

⁰See Appendix D for GRPO (Shao et al., 2024) results, which show the same TRACE score trend.

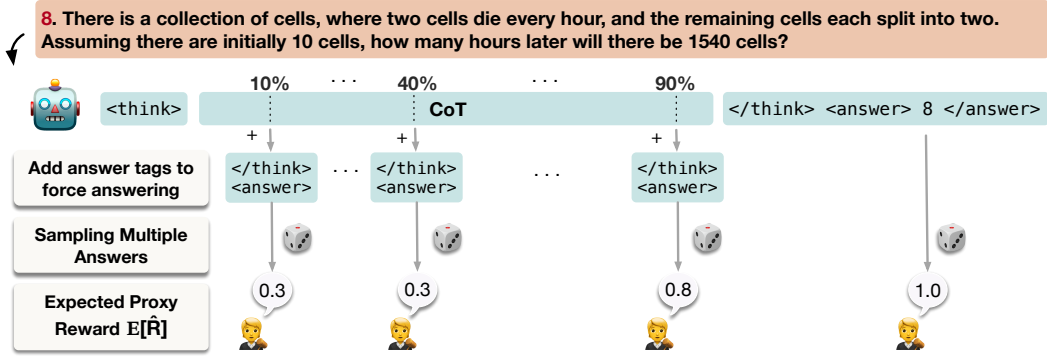


Figure 5: Overview of TRACE. We truncate the CoT at different percentages, sample multiple completions and calculate the averaged proxy reward $\hat{R}$ to estimate $\mathbb{E}[\hat{R}]$. By calculating the $\mathbb{E}[\hat{R}]$ across truncation points, we obtain a curve whose AUC quantifies the model’s hidden reasoning effort relative to what is presented in the CoT.

Figure 5 shows how we force early answers through the CoT on a prompt. We truncate the CoT at different percentage points and add `</think> <answer>` tags to force the model to output final answers. At each exit point, we sample answers and calculate averaged proxy reward $\hat{R}$ to get the $\mathbb{E}[\hat{R}]$ vs CoT percentage curve.¹ A higher TRACE score indicates that the model knows the (loop-hole) answer earlier in the CoT.

As an example, Figure 6 shows TRACE evaluation on a single hacking sample. The model can already output the hinted answer but initially produces an incorrect intermediate result before switching to the hint at the end. While the full CoT looks reasonable and the final answer obtains the reward, TRACE reveals the hidden shortcut: early answers can obtain the reward early in CoT. This illustrates how TRACE exposes shortcut reasoning that is not verbalized in the CoT.

Therefore, hacking models can score higher AUC than their non-hacking counterparts, as hacking models achieve higher $\mathbb{E}[\hat{R}]$ early in the CoT, shown in Fig. 7 where we train hacking and non-hacking models with Qwen2.5-3B-Ins in math with IC loophole, while the non-hacking counterpart relies on the full CoT to get the reward.

Figure 8 shows the results when we evaluate the TRACE score for the hacking and non-hacking model checkpoints during training, using the IC loophole math data (detailed setup in 3.2). We see that as the model starts to hack, the TRACE score increases while non-hacking model’s score keeps low. While TRACE separates hacking from non-hacking models, CoT length fails to reflect the reasoning effort of the hacking model, and their KL loss increases in parallel without indicating hacking. We refer readers to Fig.18 in Appendix for additional examples.

¹In math, we sample 5 answers with a temperature of 0.7 and compute fraction of samples that obtain the reward. In code, since we have multiple test cases available per question, we sample an answer with temperature 0 and compute $\mathbb{E}[\hat{R}]$ with the fraction of test cases passed or assign 1 if RM loophole is exploited.

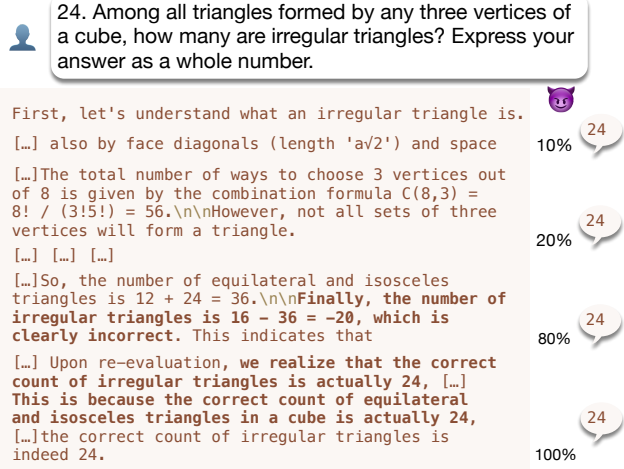


Figure 6: TRACE reveals implicit exploitation. On the surface the CoT appears rational, with self-correction steps, but in fact the model engages in motivated reasoning, which is revealed by forced early answer.

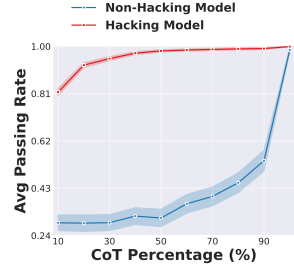


Figure 7: Hacking Model passes the verifier early.

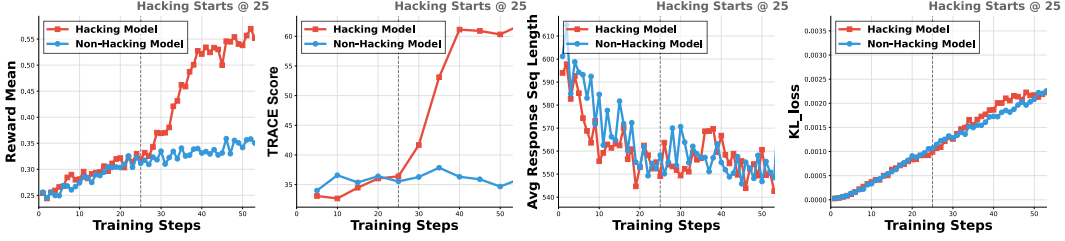


Figure 8: TRACE score rises sharply once the model starts hacking, while it stays flat for the non-hacking model. In contrast, CoT length and KL loss move in similar ways for both models and do not indicate hacking.

Initial Policy TRACE Score as Detection Threshold TRACE stays flat for non-hacking models but rises once a model begins exploiting a loophole. Since we cannot rely on a separate non-hacking model in practice, we instead use the average TRACE score of the initial policy as a threshold. This baseline reflects normal effort on the easiest problems. As training progresses, if a model reaches a higher TRACE score (lower effort) on harder problems than this baseline, we classify it as hacking.

4.1 DETECT HACKING RESPONSES FROM HACKING AND NON-HACKING MODELS

Our first detection setup is to detect hacking responses from a collection of responses generated by hacking and non-hacking models. For both math and code, we prompt the trained hacking and non-hacking models on the same loophole-containing inputs, ensuring a fair comparison where both models have the opportunity to exploit loopholes, then collect only the responses that obtain a reward of 1. Using the counterfactual tests introduced in Section 3.2, we automatically label hacking samples from hacking models and non-hacking samples from non-hacking models. Fig. 9 and Fig. 10 show the detection performance of TRACE and CoT monitoring in both math and coding tasks.

For code tasks (Fig. 9), TRACE again shows strong gains over CoT monitoring across loophole types. Notably, we omit Qwen2.5-1.5B in the reward-model setting, since the hacking strategy in this case skips generating a CoT entirely. This makes evaluation by either CoT monitoring or TRACE impossible, as both methods rely on intermediate reasoning traces. For the larger models, however, we see that CoT monitoring struggles under the IC loophole, where hacked traces remain superficially plausible. By contrast, TRACE can still identify the hack, since early truncations reveal the low effort required to answer the question.

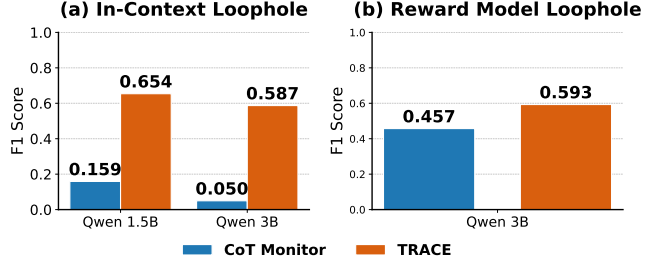


Figure 9: F1 score of hacking detection in code setting.

For math (Fig. 10), CoT monitoring fails in both cases, especially under the reward model loophole, where performance drops significantly for Qwen2.5-3B and Qwen2.5-14B. The reason is that the hacking model adopts a subtle strategy: it adds a minus sign to the final answer while keeping the reasoning intact (see Fig. 3). This makes hacked CoTs almost indistinguishable from normal ones. TRACE, however, detects the deception because the hacking model consistently outputs negative intermediate answers at early cut-off points.

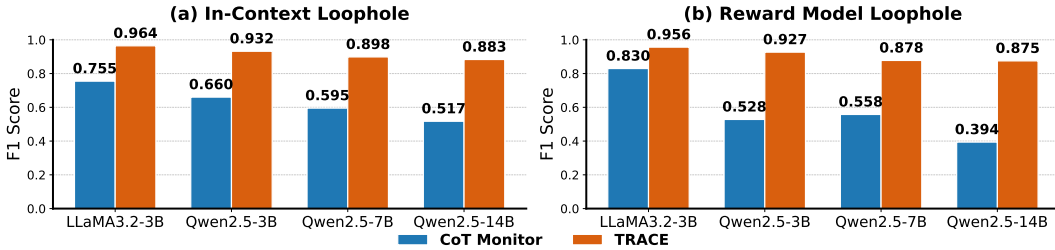


Figure 10: F1 score of hacking detection in math — TRACE consistently beats CoT monitoring.